

ASSIGNMENT NO: 2

Network Lab Experiments

Name: Akif Ayube

Class: S5 CSA

Roll No: 7

Date: 08/08/2026

Experiment 1: Hello – Client/Server Communication

Aim

To implement a basic TCP client–server application in which the client connects to the server, sends the message “Hello Server”, and receives the reply “Hello Client” from the server. This experiment demonstrates the fundamental TCP socket programming lifecycle — socket creation, binding, listening, accepting connections, and exchanging data.

Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server          (run in Terminal 1)
./client          (run in Terminal 2)
```

Procedure

Client Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9001

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
```

```

char buffer[1024] = {0};
char *message = "Hello Server";

// 1. Create socket
sock = socket(AF_INET, SOCK_STREAM, 0);

serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(PORT);
inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

// 2. Connect to server
connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
printf("Connected to server.\n");

// 3. Send message to server
send(sock, message, strlen(message), 0);
printf("Message sent to server: %s\n", message);

// 4. Receive reply from server
read(sock, buffer, 1024);
printf("Reply received from server: %s\n", buffer);

close(sock);
return 0;
}

```

Server Code

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9001

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[1024] = {0};
    char *reply = "Hello Client";

    // 1. Create socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // 2. Bind socket to IP/Port
    bind(server_fd, (struct sockaddr *)&address, sizeof(address));

    // 3. Listen for connections
    listen(server_fd, 3);
    printf("Server listening on port %d...\n", PORT);

    // 4. Accept incoming connection
    new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);
    printf("Client connected.\n");
}

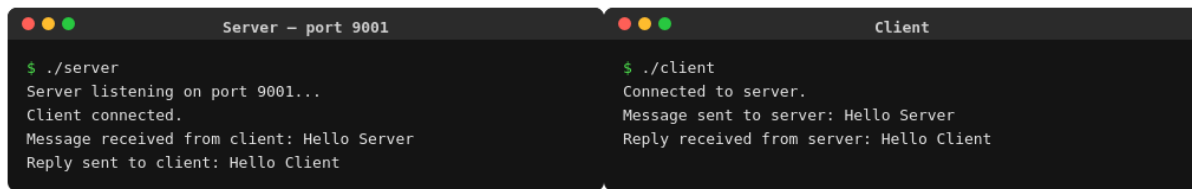
```

```
// 5. Receive message from client
read(new_socket, buffer, 1024);
printf("Message received from client: %s\n", buffer);

// 6. Send reply to client
send(new_socket, reply, strlen(reply), 0);
printf("Reply sent to client: %s\n", reply);

close(new_socket);
close(server_fd);
return 0;
}
```

Output

The image shows two side-by-side terminal windows. The left window is titled 'Server - port 9001' and shows the output of running './server'. The right window is titled 'Client' and shows the output of running './client'. Both windows show a successful connection and message exchange.

Server - port 9001	Client
<pre>\$./server Server listening on port 9001... Client connected. Message received from client: Hello Server Reply sent to client: Hello Client</pre>	<pre>\$./client Connected to server. Message sent to server: Hello Server Reply received from server: Hello Client</pre>

Conclusion

The client-server program was implemented successfully using TCP sockets. The client connected to the server, sent the message “Hello Server”, and received the reply “Hello Client”, confirming that a reliable two-way communication channel was established.

Experiment 2: Arithmetic Server

Aim

To implement a client-server application in which the client sends two integers to the server, and the server computes and returns their sum, difference, product, and quotient.

Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server          (run in Terminal 1)
./client          (run in Terminal 2)
```

Procedure

Client Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9002

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[1024] = {0};
    int nums[2];

    printf("Enter first integer: ");
    scanf("%d", &nums[0]);
    printf("Enter second integer: ");
    scanf("%d", &nums[1]);

    sock = socket(AF_INET, SOCK_STREAM, 0);

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
    printf("Connected to server.\n");

    // Send both integers to server
    send(sock, nums, sizeof(nums), 0);
    printf("Numbers sent to server.\n");

    // Receive results from server
    read(sock, buffer, 1024);
    printf("Results received from server: %s\n", buffer);

    close(sock);
    return 0;
}
```

Server Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9002

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    int nums[2];
    char result[256];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);
    printf("Server listening on port %d...\n", PORT);

    new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen);
    printf("Client connected.\n");

    // Receive two integers from client
    read(new_socket, nums, sizeof(nums));
    int a = nums[0], b = nums[1];
    printf("Received numbers: a = %d, b = %d\n", a, b);

    int sum = a + b;
    int diff = a - b;
    int prod = a * b;
    float div = b != 0 ? (float)a / b : 0;

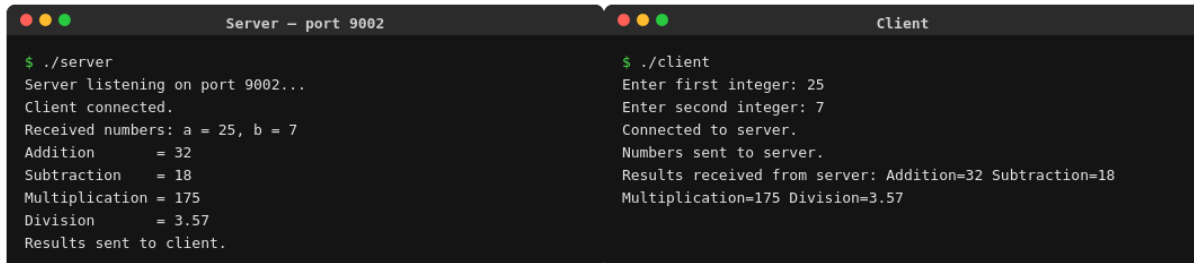
    printf("Addition      = %d\n", sum);
    printf("Subtraction    = %d\n", diff);
    printf("Multiplication = %d\n", prod);
    if (b != 0)
        printf("Division      = %.2f\n", div);
    else
        printf("Division      = undefined (divide by zero)\n");

    snprintf(result, sizeof(result),
             "Addition=%d Subtraction=%d Multiplication=%d Division=%.2f",
             sum, diff, prod, div);

    send(new_socket, result, strlen(result), 0);
    printf("Results sent to client.\n");

    close(new_socket);
    close(server_fd);
    return 0;
}
```

Output

A screenshot of two terminal windows side-by-side. The left window is titled 'Server - port 9002' and shows the execution of a server program. The right window is titled 'Client' and shows the execution of a client program. Both windows show a successful connection and the exchange of data for arithmetic operations.

```
Server - port 9002
$ ./server
Server listening on port 9002...
Client connected.
Received numbers: a = 25, b = 7
Addition      = 32
Subtraction   = 18
Multiplication = 175
Division      = 3.57
Results sent to client.

Client
$ ./client
Enter first integer: 25
Enter second integer: 7
Connected to server.
Numbers sent to server.
Results received from server: Addition=32 Subtraction=18
Multiplication=175 Division=3.57
```

Conclusion

The arithmetic server was successfully implemented. For the input values 25 and 7, the server correctly computed Addition = 32, Subtraction = 18, Multiplication = 175, and Division = 3.57, and returned the results to the client over the socket connection.

Experiment 3: String Processing Server

Aim

To implement a client-server application in which the client sends a string to the server, and the server computes the length of the string, converts it to uppercase, and reverses it, then returns the results to the client.

Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server          (run in Terminal 1)
./client          (run in Terminal 2)
```

Procedure

Client Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9003

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[1024] = {0};
    char str[256];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0'; // remove trailing newline

    sock = socket(AF_INET, SOCK_STREAM, 0);

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
    printf("Connected to server.\n");

    // Send string to server
    send(sock, str, strlen(str) + 1, 0);
    printf("String sent to server.\n");

    // Receive processed results from server
    read(sock, buffer, 1024);
    printf("Results received from server: %s\n", buffer);

    close(sock);
    return 0;
}
```

Server Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9003

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char str[256];
    char upper[256];
    char rev[256];
    char result[1024];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);
    printf("Server listening on port %d...\n", PORT);

    new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);
    printf("Client connected.\n");

    // Receive string from client
    memset(str, 0, sizeof(str));
    read(new_socket, str, sizeof(str));
    printf("Received string: %s\n", str);

    int len = strlen(str);

    // Convert to uppercase
    for (int i = 0; i <= len; i++)
        upper[i] = toupper(str[i]);

    // Reverse string
    for (int i = 0; i < len; i++)
        rev[i] = str[len - 1 - i];
    rev[len] = '\0';

    printf("Length      = %d\n", len);
    printf("Uppercase    = %s\n", upper);
    printf("Reversed     = %s\n", rev);

    snprintf(result, sizeof(result), "Length=%d Uppercase=%s Reversed=%s",
             len, upper, rev);

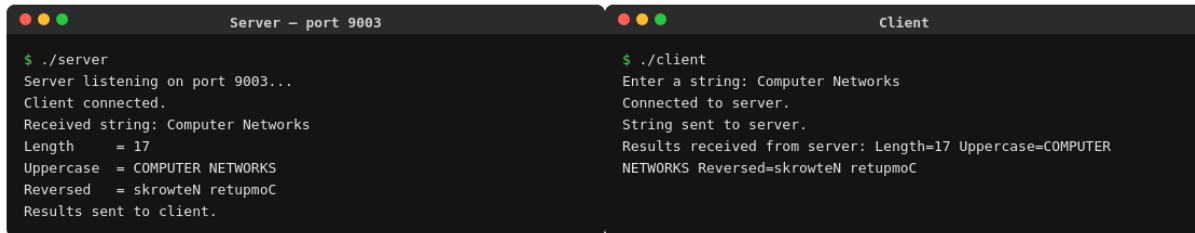
    send(new_socket, result, strlen(result), 0);
    printf("Results sent to client.\n");

    close(new_socket);
    close(server_fd);
}
```



```
}    return 0;
```

Output



```
Server - port 9003
$ ./server
Server listening on port 9003...
Client connected.
Received string: Computer Networks
Length      = 17
Uppercase   = COMPUTER NETWORKS
Reversed    = skrowteN retupmoC
Results sent to client.

Client
$ ./client
Enter a string: Computer Networks
Connected to server.
String sent to server.
Results received from server: Length=17 Uppercase=COMPUTER
NETWORKS Reversed=skrowteN retupmoC
```

Conclusion

The string processing server was implemented and tested with the input “Computer Networks”. The server correctly returned Length = 17, Uppercase = COMPUTER NETWORKS, and Reversed = skrowteN retupmoC, demonstrating successful string manipulation over a socket connection.

Experiment 4: Array Processing Server

Aim

To implement a client-server application in which the client sends an array of N integers to the server, and the server computes the sum, average, maximum, and minimum of the array elements.

Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server          (run in Terminal 1)
./client          (run in Terminal 2)
```

Procedure

Client Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9004

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[1024] = {0};
    int n;
    int arr[100];

    printf("Enter number of elements (N): ");
    scanf("%d", &n);
    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    sock = socket(AF_INET, SOCK_STREAM, 0);

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
    printf("Connected to server.\n");

    // Send N followed by the array
    send(sock, &n, sizeof(int), 0);
    send(sock, arr, n * sizeof(int), 0);
    printf("Array sent to server.\n");

    // Receive results from server
    read(sock, buffer, 1024);
    printf("Results received from server: %s\n", buffer);

    close(sock);
}
```

```
    return 0;
}
```

Server Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9004

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    int n;
    int arr[100];
    char result[512];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);
    printf("Server listening on port %d...\n", PORT);

    new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);
    printf("Client connected.\n");

    // Receive N
    read(new_socket, &n, sizeof(int));
    // Receive array of N integers
    read(new_socket, arr, n * sizeof(int));

    printf("Received array of %d elements: ", n);
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");

    int sum = 0, maxVal = arr[0], minVal = arr[0];
    for (int i = 0; i < n; i++) {
        sum += arr[i];
        if (arr[i] > maxVal) maxVal = arr[i];
        if (arr[i] < minVal) minVal = arr[i];
    }
    float avg = (float)sum / n;

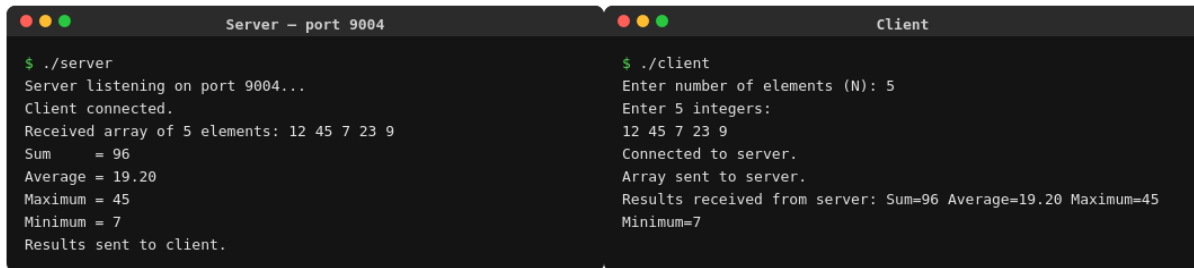
    printf("Sum      = %d\n", sum);
    printf("Average = %.2f\n", avg);
    printf("Maximum = %d\n", maxVal);
    printf("Minimum = %d\n", minVal);

    snprintf(result, sizeof(result), "Sum=%d Average=%.2f Maximum=%d Minimum=%d",
             sum, avg, maxVal, minVal);

    send(new_socket, result, strlen(result), 0);
    printf("Results sent to client.\n");
}
```

```
close(new_socket);  
close(server_fd);  
return 0;  
}
```

Output



The screenshot shows two terminal windows side-by-side. The left window is titled 'Server - port 9004' and the right window is titled 'Client'. The server window shows the execution of './server', which starts listening on port 9004, receives a connection, and processes an array of 5 integers (12, 45, 7, 23, 9). It calculates the sum (96), average (19.20), maximum (45), and minimum (7), and sends these results to the client. The client window shows the execution of './client', which prompts for the number of elements (5) and the integers (12, 45, 7, 23, 9). It then receives the results from the server: Sum=96, Average=19.20, Maximum=45, and Minimum=7.

```
Server - port 9004  
$ ./server  
Server listening on port 9004...  
Client connected.  
Received array of 5 elements: 12 45 7 23 9  
Sum      = 96  
Average = 19.20  
Maximum  = 45  
Minimum  = 7  
Results sent to client.  
  
Client  
$ ./client  
Enter number of elements (N): 5  
Enter 5 integers:  
12 45 7 23 9  
Connected to server.  
Array sent to server.  
Results received from server: Sum=96 Average=19.20 Maximum=45  
Minimum=7
```

Conclusion

The array processing server was successfully implemented and tested with the array {12, 45, 7, 23, 9}. The server correctly computed Sum = 96, Average = 19.20, Maximum = 45, and Minimum = 7, verifying correct array handling over the client-server model.

Experiment 5: Matrix Analyzer

Aim

To implement a client-server application in which the client sends a square matrix to the server, and the server determines the type of matrix (diagonal, upper triangular, lower triangular, or general), and computes the sum of its elements and its trace.

Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server          (run in Terminal 1)
./client          (run in Terminal 2)
```

Procedure

Client Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9005
#define MAXN 20

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[1024] = {0};
    int n;
    int mat[MAXN * MAXN];    // flat array, indexed as mat[i*n + j]

    printf("Enter order of square matrix (N): ");
    scanf("%d", &n);
    printf("Enter %d x %d matrix elements:\n", n, n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &mat[i * n + j]);

    sock = socket(AF_INET, SOCK_STREAM, 0);

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
    printf("Connected to server.\n");

    // Send N followed by the matrix
    send(sock, &n, sizeof(int), 0);
    send(sock, mat, n * n * sizeof(int), 0);
    printf("Matrix sent to server.\n");

    // Receive results from server
```

```

    read(sock, buffer, 1024);
    printf("Results received from server: %s\n", buffer);

    close(sock);
    return 0;
}

```

Server Code

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9005
#define MAXN 20

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    int n;
    int mat[MAXN * MAXN]; // flat array, indexed as mat[i*n + j]
    char result[512];
    char type[50];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }
    listen(server_fd, 3);
    printf("Server listening on port %d...\n", PORT);
    fflush(stdout);

    new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen);
    printf("Client connected.\n");

    // Receive N
    read(new_socket, &n, sizeof(int));
    // Receive N*N matrix elements (row-major, contiguous)
    read(new_socket, mat, n * n * sizeof(int));

    printf("Received %dx%d matrix:\n", n, n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%4d", mat[i * n + j]);
        printf("\n");
    }

    int sum = 0, trace = 0;
    int isDiagonal = 1, isUpper = 1, isLower = 1;

```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        int val = mat[i * n + j];
        sum += val;
        if (i == j) trace += val;
        if (i != j && val != 0) {
            isDiagonal = 0;
            if (i > j) isUpper = 0;    // element below diagonal is non-zero
            if (i < j) isLower = 0;    // element above diagonal is non-zero
        }
    }
}

if (isDiagonal)
    strcpy(type, "Diagonal Matrix");
else if (isUpper)
    strcpy(type, "Upper Triangular Matrix");
else if (isLower)
    strcpy(type, "Lower Triangular Matrix");
else
    strcpy(type, "General Matrix");

printf("Matrix Type = %s\n", type);
printf("Sum of Elements = %d\n", sum);
printf("Trace = %d\n", trace);

snprintf(result, sizeof(result), "MatrixType=%s Sum=%d Trace=%d",
        type, sum, trace);

send(new_socket, result, strlen(result), 0);
printf("Results sent to client.\n");

close(new_socket);
close(server_fd);
return 0;
}

```

Output

```

Server - port 9005
$ ./server
Server listening on port 9005...
Client connected.
Received 3x3 matrix:
 2  0  0
 0  5  0
 0  0  9
Matrix Type = Diagonal Matrix
Sum of Elements = 16
Trace = 16
Results sent to client.

Client
$ ./client
Enter order of square matrix (N): 3
Enter 3 x 3 matrix elements:
2 0 0
0 5 0
0 0 9
Connected to server.
Matrix sent to server.
Results received from server: MatrixType=Diagonal Matrix Sum=16
Trace=16

```

Conclusion

The matrix analyzer server was successfully implemented and tested with a 3x3 diagonal matrix. The server correctly identified the matrix type as “Diagonal Matrix”, and computed Sum of Elements = 16 and Trace = 16, confirming correct matrix analysis over the socket connection.